Proof Artifact Report for the HAETAE NTT EasyCrypt Development

Artifact documentation for the `proof/` directory

April 2, 2026

Abstract

This report documents the EasyCrypt proof artifact currently stored in the `proof/` directory of the HAETAE NTT repository. The artifact aims to connect a Jasmin-extracted implementation of HAETAE polynomial arithmetic to coefficient-level specifications over the finite field $\mathbb{Z}_{64513}$. The development follows the proof architecture used in the ML-KEM/Kyber artifacts, but it is adapted to HAETAE's modulus, 32-bit Montgomery arithmetic, and 256-coefficient polynomial layout. The report explains the role of each proof module, the main definitions and theorems, the current proof status, and the operational checks available to test the artifact.

1 Artifact Scope

The `proof/` directory contains six substantive EasyCrypt modules and three small array-wrapper files:

- `HAETAE_poly_extract.ec`: extracted EasyCrypt model of the Jasmin code, including the concrete twiddle tables and the imperative procedures for Montgomery reduction, field multiplication, NTT, inverse NTT, and coefficient-wise base multiplication.
- `HAETAE_GFq.ec`: finite-field model for HAETAE's modulus $q = 64513$.
- `HAETAE_Montgomery.ec`: generic signed Montgomery reduction theory, instantiated later with the HAETAE parameters.
- `HAETAE_FCLib.ec`: low-level word lemmas needed to bridge Jasmin bit-vector operations with mathematical integer semantics.
- `HAETAE_Fq.ec`: instantiation of Montgomery arithmetic for the extracted `__montgomery_reduce` and `__fqmul` routines.
- `HAETAE_poly.ec`: abstract NTT specification, losslessness proofs, representation bridges, relational correctness statements, and wrapper correctness theorems.

The auxiliary files `Array256.ec`, `BArray1024.ec`, and `WArray1024.ec` provide array interfaces used by the extracted code and its proofs. The corresponding `.eco` files are compiled EasyCrypt artifacts that speed up replay and serve as evidence that individual modules have been checked in the local toolchain.

2 Design Overview

The proof development is layered. Each layer eliminates a different gap between the extracted implementation and the target mathematical statement.

2.1 Layer 1: Extracted Program Model

`HAETAE_poly_extract.ec` defines `module M`, which is the concrete program artifact to be verified. Its key procedures are:

- `M.__montgomery_reduce`: reduces a signed 64-bit intermediate to a 32-bit coefficient using HAETAE's Montgomery parameters.
- `M.__fqmul`: multiplies two signed 32-bit coefficients and applies Montgomery reduction.
- `M._poly_ntt`: iterative breadth-first NTT on 256 coefficients using concrete twiddle tables `jzetas`.
- `M._poly_invntt`: inverse NTT using `jzetas_inv` followed by final scaling.
- `M._poly_basemul`: coefficient-wise multiplication in the Montgomery domain.
- Wrapper procedures `poly_ntt_jazz`, `poly_invntt_jazz`, and `poly_basemul_jazz`.

This file is intentionally operational: it contains the concrete control flow, loop counters, word-level loads/stores, and precomputed constants that later proof layers must interpret.

2.2 Layer 2: Algebraic Model

`HAETAE_GFq.ec` defines the coefficient type by cloning EasyCrypt's `ZModField` at modulus 64513. The file also introduces the signed projection `as_sint`, which is used throughout the artifact to express range bounds on coefficients.

`HAETAE_Montgomery.ec` then develops a parameterized theory `SignedReductions`. The central object is the signed Montgomery reduction operator

$$\texttt{SREDC}(a),$$

whose main theorem `SREDCp_corr` proves both a range bound

$$-q \leq \texttt{SREDC}(a) < q$$

under the standard input precondition, and the expected modular congruence

$$\texttt{SREDC}(a) \equiv a \cdot R^{-1} \pmod{q}.$$

This generic theorem is the algebraic backbone for the later correctness proofs of HAETAE's concrete field multiplication.

2.3 Layer 3: Word-Level Bridging

`HAETAE_FCLib.ec` contains the bit-level lemmas needed to connect the Jasmin right-shift and sign-extension operators to integer division. The most important theorem is `SAR_sem32`, which is intended to justify that arithmetic right shift by 32 bits matches signed division by 2^{32}. This is precisely the semantic bridge required by the extracted Montgomery reduction code.

2.4 Layer 4: Field Arithmetic Correctness

`HAETAE_Fq.ec` instantiates `SignedReductions` with the HAETAE parameters:

$$q = 64513, \qquad R = 2^{32}, \qquad q^{-1} \bmod R = 940508161, \qquad R^{-1} \bmod q = 50386.$$

It then states the two key arithmetic correctness lemmas:

- `montgomery_reduce_corr_h`: correctness of `M.__montgomery_reduce`.
- `fqmul_corr_h`: correctness of `M.__fqmul`.

These theorems are lifted into probability-1 judgments (`phoare`) by combining them with losslessness results.

2.5 Layer 5: Polynomial-Level Correctness

`HAETAE_poly.ec` is the main algorithmic proof file. It introduces:

- an abstract coefficient-level `NTT` module with procedures `ntt`, `invntt`, and `basemul`;
- losslessness theorems for the abstract algorithms and the extracted implementations;
- representation bridges such as `lift_array256`, `zetas_coeff`, `zetas_inv_coeff`, and `signed_bound_cxq`;
- algebraic bridge lemmas `rrinvcoeff` and `mul_congr`;
- relational correctness statements `poly_ntt_equiv`, `poly_invntt_equiv`, and `poly_basemul_equiv`;
- wrapper correctness statements for the public entry points.

This file is where the ML-KEM proof structure is being ported to HAETAE. The current `poly_ntt_equiv` statement already carries a stage-sensitive bound invariant: a signed-q input bound is tracked to a signed-$9q$ output bound through the eight NTT stages.

3 Module-by-Module Inventory

Module	Role	Key objects and theorems	Current status
`HAETAE_GFq.ec`	Finite-field interface	`q`, `prime_q`, clone of `ZModField`, signed projection `as_sint`	Complete in current file
`HAETAE_Montgomery.ec`	Generic signed Montgomery theory	`SignedReductions`, `smod`, `SREDC`, `SREDCp_corr`	Core theorem complete in current file
`HAETAE_FCLib.ec`	Word-level helper lemmas	`mask32_hi`, `aux32_0`, `aux32_1`, `aux32_2`, `SAR_sem32`	One admitted lemma
`HAETAE_Fq.ec`	Concrete arithmetic correctness	`montgomery_reduce_corr_h`, `montgomery_reduce_corr`, `fqmul_corr_h`, `fqmul_corr`	Two admitted lemmas
`HAETAE_poly_extract.ec`	Extracted imperative model	`module M`, `jzetas`, `jzetas_inv`, `_poly_ntt`, `_poly_invntt`, `_poly_basemul`	Operational model present
`HAETAE_poly.ec`	Abstract spec and relational correctness	`module NTT`, `lift_array256`, `signed_bound_cxq`, `poly_ntt_equiv`, `poly_invntt_equiv`, `poly_basemul_equiv`	Nine admitted lemmas

4 Main Mathematical Content

4.1 Finite-Field and Representation Bridge

The coefficient type is represented abstractly as elements of $\mathbb{Z}_{64513}$ and concretely as signed 32-bit integers stored in `BArray1024`. The bridge

$$\texttt{lift_array256}(p) = \texttt{Array256.init}(\lambda i.\,\texttt{incoeff}(\texttt{W32.to_sint}(\texttt{get32}\ p\ i)))$$

maps a concrete polynomial buffer to the abstract coefficient vector consumed by the specification. This is the canonical relation used in all equivalence theorems.

The twiddle tables require one additional adjustment. Since the extracted code multiplies through `__fqmul`, the concrete table entries are interpreted in the proof as

$$\texttt{incoeff}(\text{table entry}) \cdot R^{-1},$$

captured by `zetas_coeff` and `zetas_inv_coeff`. The lemma `rrinvcoeff` is then used to cancel the extra Montgomery factor.

4.2 Losslessness

Before relational correctness is attempted, the artifact establishes losslessness of both the abstract algorithms and their extracted counterparts. This matters for two reasons:

- it guarantees that the imperative loops are structurally well-founded in EasyCrypt's semantics;
- it provides the probabilistic side conditions needed to lift Hoare-style arithmetic results into probability-1 correctness statements.

The files `HAETAE_poly.ec` and `HAETAE_Fq.ec` already contain completed losslessness lemmas such as `ntt_ll`, `poly_ntt_ll`, `montgomery_reduce_ll`, and `fqmul_ll`.

4.3 Montgomery Arithmetic

The intended proof story for field arithmetic is standard:

1. prove that the extracted right shift and truncation operations implement the integer divisions and modular reductions appearing in the definition of `SREDC`;
2. instantiate the generic theorem `SREDCp_corr`;
3. conclude that `__fqmul` computes multiplication in the Montgomery domain with the expected R^{-1} factor.

This bridge is already structurally in place, but its last steps still depend on the admitted word-level and arithmetic lemmas in `HAETAE_FCLib.ec` and `HAETAE_Fq.ec`.

4.4 NTT-Level Correctness

The central algorithmic theorem is currently:

```
equiv poly_ntt_equiv :
  NTT.ntt ~ M._poly_ntt :
    r{1} = lift_array256 rp{2} /\
    zetas{1} = zetas_coeff /\
    signed_bound_cxq rp{2} 0 256 1
    ==>
    res{1} = lift_array256 res{2} /\
    signed_bound_cxq res{2} 0 256 9.
```

This statement is important for paper exposition because it shows the exact contract carried by the ported ML-KEM invariant:

- relational equality between the abstract NTT state and the lifted concrete state;

- stage-sensitive signed bounds on the concrete representation;
- explicit alignment between the extracted twiddle table and the abstract coefficient table.

The proof skeleton in `HAETAE_poly.ec` already identifies the intended HAETAE analogues of the ML-KEM ingredients:

- `signed_bound_cxq` for concrete coefficient range tracking;
- `zeta_bound` for twiddle-table range facts;
- `rrinvcoeff` to eliminate the residual Montgomery factor;
- `mul_congr` to convert concrete integer multiplication into abstract coefficient multiplication.

5 Current Proof Gaps

At the time of writing, the artifact still contains twelve explicit `admit.` placeholders:

- `HAETAE_FCLib.ec`: `SAR_sem32`.
- `HAETAE_Fq.ec`: `montgomery_reduce_corr_h`, `fqmul_corr_h`.
- `HAETAE_poly.ec`: `zeta_bound`, `zetainv_bound`, `ntt_inner_loop_correct`, `poly_ntt_equiv`, `poly_ntt_corr`, `poly_invntt_equiv`, `poly_invntt_corr`, `poly_basemul_equiv`, `poly_basemul_corr`.

These gaps are not cosmetic; they identify the precise frontier between the already-ported framework and the remaining mechanization work.

5.1 What Is Already Established

Several important components are already in place:

- the finite-field model and parameter instantiation;
- the generic signed Montgomery correctness theorem;
- the extracted imperative model and all concrete constants;
- losslessness proofs for the abstract and concrete procedures;
- the shape of the main relational theorems and the HAETAE-specific NTT bound invariant.

5.2 What Still Blocks Full End-to-End Correctness

The remaining work clusters into three proof-critical areas:

1. **Word semantics.** The negative-case proof of `SAR_sem32` is still missing.
2. **Field arithmetic.** The high-level correctness of `__montgomery_reduce` and `__fqmul` is stated but not yet fully discharged.
3. **Algorithmic relational proofs.** The NTT, inverse NTT, and base multiplication equivalence proofs are scaffolded but still admitted.

6 Operational Validation

The repository now includes a dedicated artifact test runner:

`tools/test-proof-artifacts.sh`

and matching `Makefile` targets:

- `make test`
- `make proof-test`
- `make proof-test-quiet`
- `make proof-test-strict`

The test runner performs the following checks:

1. command availability (`easycrypt`, `why3`, `why3server`);
2. presence of required `.ec` and `.eco` artifacts;
3. freshness of compiled `.eco` files relative to their sources;
4. inventory of unresolved `admit.` placeholders;
5. staged EasyCrypt smoke compiles for the proof modules.

In the current local environment, the script reports an `ENVIRONMENT_FAILURE` because `why3server` is not available in `PATH`. This is an environment issue rather than a mathematical contradiction in the proof artifact. The same run also reports that `HAETAE_poly.eco` is stale relative to `HAETAE_poly.ec`, which is expected after recent edits to the NTT invariant.

7 How to Cite the Artifact State in a Paper

For a paper or technical report, the artifact can be described accurately as follows:

> The EasyCrypt development contains a complete extracted model of HAETAE's polynomial arithmetic, a finite-field formalization at $q = 64513$, a parameterized Montgomery reduction theory, completed losslessness proofs for the abstract and extracted procedures, and a partially mechanized port of the ML-KEM relational proof architecture for NTT correctness. The remaining work is localized to the word-level shift semantics, the final field-arithmetic bridge lemmas, and the admitted relational correctness proofs for NTT, inverse NTT, and base multiplication.

This wording is precise about what has been formalized, what has been proved, and what remains unfinished.

8 Conclusion

The `proof/` artifact is already organized as a serious verification development rather than a flat collection of proof scripts. Its strongest features are the clean separation between extracted code, algebraic theory, and algorithmic correctness, together with the explicit reuse of the ML-KEM proof structure. Its current limitations are also sharply localized and therefore tractable. From a paper-writing perspective, this makes the artifact mature enough to describe in detail, provided the write-up distinguishes clearly between completed layers, admitted lemmas, and environment-dependent replay requirements.